

FreCoS: A Learned Staleness Model for Semantic Caches

A freshness- and cost-aware extension to GPTCache

Lior Lotan

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev

July 25, 2026

Abstract

Semantic caches are evaluated on hit rate, latency, and at best false-hit rate, none of which distinguishes a correct hit from a stale one. We name and measure that gap directly: *stale-hit-rate*, the fraction of served cache hits whose content was no longer valid at serve time. We extend GPTCache, a frozen and widely used open-source semantic cache, with a per-cluster staleness model that is fit from observed data and consumed in two places: a validity gate on the serve path, and a soft decay term in eviction. Across a bracketing experiment, a six-row ablation, and three parameter sweeps, all run with ten seeds per configuration on a deterministic synthetic workload, we find that the validity gate cuts stale-hit-rate by roughly fifteen times relative to a frequency-based floor, that the learned decay rate reliably outperforms a single pooled rate and tracks an oracle rate closely, and that this correctness gain comes at a bounded and quantified cost in hit rate. We report the size of that trade-off directly rather than presenting the extension as a uniform win.

1 Introduction

1.1 Motivation

A semantic cache sits in front of an expensive backend, usually a large language model call, and serves a stored response when an incoming query is close enough in embedding space to one it has already answered. The promise is straightforward: skip the call, return the same answer faster and for free. The problem is that “the same answer” is a claim about time as well as similarity, and most semantic caches never check the time part.

This gap is not a corner case. It sits at the center of a growing class of production systems that place one or more large language models between a user and an answer about frequently changing content. Consider a system whose job is to monitor how a brand is represented when people ask AI assistants and search engines about it, answering questions such as “what do shoppers see when they ask an AI assistant about this product line” many times a day, across many brands, many geographies, and many underlying language models — the kind of workload behind answer engine optimization (AEO), a discipline that has emerged specifically because brand visibility inside AI-generated answers, rather than inside a ranked list of links, is now a measurable and contested surface. The set of distinct questions such a system fields is highly repetitive: the same style of question, sometimes the identical phrasing, is asked about the same brand or product category over and over as new users are onboarded or as monitoring jobs re-run on a schedule. That repetition is exactly the shape of workload a semantic cache is built to absorb, and absorbing it cheaply matters, since every query answered by a language model carries a real, per-call dollar cost that scales with the number of brands and questions tracked.

The catch is that the underlying facts these questions are about do not sit still. A product goes out of stock, a price changes, a competitor launches, a review’s sentiment shifts, and the correct answer to an unchanged question changes with it. A cache that only asks “have I seen

a similar enough question before” and never asks “is that answer still true” will happily keep serving a stale answer indefinitely once it has been written, and the system built on top of it has no way to distinguish a correct cache hit from an incorrect one, because the metric that would distinguish them, stale-hit-rate, does not exist in any standard cache evaluation. This is the general shape of the problem this project addresses: answer-facing systems built on top of language models need a caching layer that treats correctness over time as a first-class property, not an afterthought bolted onto count-based eviction.

Two concrete gaps follow from this observation, and they motivate the two components this project adds.

No staleness concept in the serve path. A cached answer about a fact that changes on its own schedule is served indefinitely once written. “Found in cache” is silently treated as “correct,” with no mechanism to ever reconsider that judgment as time passes.

No cost term in eviction. Count-based eviction policies such as LRU and LFU treat two entries with identical access patterns identically, even when one is dramatically more expensive to regenerate than the other. A policy blind to regeneration cost will happily evict a rarely accessed but very expensive entry ahead of a cheap one accessed once more recently, which is the wrong trade from a cost-minimization standpoint.

The current state of both industry and research reflects this gap unevenly. Frontier model providers do exact-prefix key-value caching with a wall-clock time-to-live and no similarity matching at all, which sidesteps the freshness question by making every cache key an exact match. Semantic-caching products in the open-source and commercial space use a single fixed global similarity threshold with plain time-to-live or count-based eviction, with no notion of per-topic staleness. The research literature, discussed next, is further ahead than either of these, having proposed cost-aware and even staleness-aware eviction formulas, but none of it names or measures stale-hit-rate as an evaluation metric in its own right. This project positions itself against that literature, not against the weaker product baseline.

1.2 Related work

The closest prior art, and this project’s primary eviction comparator, is Biton and Friedman’s work on semantic-cache eviction[1]. The paper proves that optimal offline semantic-cache eviction is NP-hard, supplies polynomial-time heuristics, and presents online eviction policies that combine recency, frequency, and locality signals. It evaluates these policies on GPTCache and releases its implementation, which is part of why GPTCache was selected as this project’s baseline in the first place. Two findings from that paper shape this project’s experiment design directly: the authors show that LRU is a weak eviction policy on most semantic caching workloads and that frequency-based policies are a much stronger baseline, which is why every ablation and comparison in this report treats an LFU-class policy, not LRU, as the floor of comparison. The delta against FreCoS is specific: their policies combine access-pattern signals only, recency, frequency, and locality, while this project adds two signals orthogonal to access pattern, regeneration cost and content validity over time, and evaluates against a correctness metric, stale-hit-rate, that their setting does not define at all.

The structurally closest published eviction formula is Cortex’s LCFU policy[2], whose earlier preprint circulated under the name Asteria and is cited here by its current title. LCFU scores a cache entry as a product of log-scaled frequency, cost, latency, and a staleness term, divided by size, combined with a time-to-live purge. The delta versus FreCoS is narrow: Cortex’s staleness term is a static “staticity” score from one to ten, assigned by a language model at write time and paired with a user-defined time-to-live, while FreCoS’s staleness term is a decay rate learned per cluster from observed staleness in a held-out calibration split. The distinction is learned against assigned, and the bracketing experiment in Section 5.1 is built specifically to test whether that learning does measurable work. Cortex’s LCFU formula was not reimplemented

as a comparator, since it incorporates remote-tool-call latency metadata that has no equivalent in this project’s setting.

Several other lines of work bear on individual pieces of the design without matching its combination. GDSF[3] scores entries as cost times frequency divided by size, with a global-clock aging term, but that aging term ages access recency, not content validity, so it has no freshness semantics at all. Category-Aware caching[4] assigns per-category time-to-live values that are load-based rather than learned from observed staleness, and defines no stale-hit-rate equivalent; it is also the reason a third, originally planned component of this project, calibrated per-cluster similarity thresholds, was cut, since that component would have been a lossy special case of both this work and per-embedding threshold learning. FreshCache[5] shares this project’s lineage of putting a staleness concept directly in the serve path, but treats it as a probabilistic risk budget per tier rather than the hard learned gate used here, and is listed as grounded future work as an alternative framing. SCALM[6] clusters cache entries in a way structurally similar to this project’s k-means step, but its per-cluster significance score is not staleness-aware. MeanCache[7] is cost-motivated but explicitly does not target eviction. W-TinyLFU[8] is an admission-control policy, deciding whether a new entry is worth admitting at all before eviction is ever invoked, and is out of scope here since porting it would add an orthogonal axis of improvement rather than test either of this project’s two contributions.

The alternative baseline vCache[9] was considered and rejected. Its contribution is per-prompt similarity threshold learning, replacing a single global threshold with one learned per cached embedding, and eviction is explicitly not its focus. Building this project’s staleness and cost work on top of vCache would mean layering a new contribution on a system whose own novelty sits in a different part of the pipeline, diluting rather than sharpening the comparison.

2 Baseline

2.1 Why GPTCache

This project extends GPTCache[10] rather than a more actively maintained semantic cache, and the case rests on architecture and control properties rather than on community activity. GPTCache’s most recent release is v0.1.44 (August 2024); at the time of writing it carries dozens of open issues and pull requests, and the maintainers’ own documentation states that support for new model and API integrations has stopped. Ordinarily this would count against a dependency. Here it is treated as an advantage: a frozen application programming interface is a better scientific control than a moving one, since the baseline measured early in this project is byte-identical to the baseline measured at the end, and every performance delta reported below is attributable to the extension rather than to an upstream change landing mid-project.

The second reason is architectural fit, verified by reading the vendored source directly rather than assumed from documentation. Every component this project modifies, the similarity threshold check, the hit and miss decision, and the eviction policy registry, sits behind a registered, named interface, so the extension is additive configuration: a new eviction subclass registered through the existing hook, a metadata field added via composition around the storage layer, and a serving-path gate that plugs into one shared decision helper. Nothing vendored is edited. Every specific claim about GPTCache’s internal behavior quoted in this report was re-verified against the exact pinned commit vendored in this project’s repository, with the file and line number recorded, rather than carried over unverified from an earlier reading of the source.

Third, GPTCache is the baseline the closest prior art already uses, as noted above, which makes Biton and Friedman’s released eviction policy usable directly as this project’s primary comparator rather than requiring a reimplementaion on a different substrate. Fourth, GPTCache’s default embedding backend runs on the CPU and deterministically, which keeps every

benchmark run reproducible without a graphics processing unit dependency and without non-determinism introduced by a hosted embedding service.

2.2 GPTCache’s behavior and default eviction policy

GPTCache sits in front of a language model call and serves a cached response when an incoming query is semantically close enough to a previously seen one, using a single global cosine similarity threshold, 0.8 by default, rather than exact-match keys. Its eviction policies are purely count-based: least-recently-used, least-frequently-used, first-in-first-out, or random replacement, selected by name at construction time. None of them reads generation time, access recency, or regeneration cost as anything beyond the single counter each policy already tracks, and none retains any metadata on an evicted entry; the underlying eviction cache stores only a boolean flag per key. Creation and last-access timestamps are persisted per entry but are never read by any decision logic in the adapter or the eviction policies; they exist as columns with no downstream consumer, which is precisely the gap this project’s extension repurposes. There is no admission control, every miss is written back unconditionally, and no cost accounting, false-hit tracking, or staleness tracking exists anywhere in the built-in reporting layer, which counts only per-stage operation timers.

3 Design

3.1 Contribution

The contribution is one learned artifact with two consumers. FreCoS fits a per-cluster staleness model and uses it at both ends of the cache’s lifecycle: as a hard validity gate when serving a candidate hit, and as a soft decay term when choosing an eviction victim. The metric that makes both uses visible is stale-hit-rate itself, defined as the fraction of served hits whose content was already invalid at the moment it was served.

Cluster assignment is a fixed, seeded k-means clustering over cached-query embeddings, run once at cache-build time. There is no online re-clustering; this is a deliberate reproducibility choice, since dynamic re-clustering would make repeated runs of the same configuration non-deterministic. Cluster identity, once assigned, is consumed by both the gate and the eviction policy and is never recomputed independently by either one, which matters because a gate and an eviction policy that disagree about which cluster an entry belongs to is one of the most likely integration failures in a system with this shape.

For each cluster, a decay rate is fit from the observed staleness of entries in a held-out calibration split, and a time-to-live is derived from that rate at a configured confidence level. Three interchangeable modes produce this per-cluster table with the same output shape: a *learned* mode that fits each cluster’s rate independently from its own observations, falling back to a single pooled rate when a cluster has fewer than thirty observations; a *global* mode that pools every cluster’s observations into one shared rate; and an *oracle* mode that takes the rate directly from ground truth, used only as an upper bound for evaluation, never available to the system in a real deployment.

The validity gate is the first consumer. On a candidate hit, if the age of the cached entry exceeds its cluster’s time-to-live, the entry is treated as a miss rather than served, the answer is regenerated, and the event is counted as a stale hit prevented rather than a stale hit served. This is a single comparison against the entry’s creation time; the gate never reads the entry’s last-access time, since content staleness is a property of when an answer was generated, not of how recently it happened to be requested, and a popular but outdated entry must not be treated as fresh simply because it is popular.

The eviction value function is the second consumer:

$$\text{value} = \log(2 + \text{freq}) \cdot \log(1 + \kappa \cdot \text{regen_cost}) \cdot e^{-\lambda_c \cdot \text{age}} / \text{size_bytes} \quad (1)$$

where age is measured from creation time, never from last access, for the same reason the gate uses creation time; κ is a fixed constant that log-compresses the regeneration cost so that a heavy-tailed cost distribution cannot dominate the other three terms; and the victim chosen for eviction is the entry with the lowest value, with ties broken deterministically by oldest creation time and then by lowest entry identifier.

One implementation detail is worth stating plainly because it reflects a real design tension the project encountered rather than a footnote. The design’s original frequency term was $\log(1 + \text{freq})$, which is exactly zero when an entry is brand new and has never been accessed. Because the value function is a product of four terms, a zero frequency term forces the entire score to zero regardless of how expensive, fresh, or small the entry is, which means a freshly written entry would always be the eviction victim on the very next insert. This was caught by a cold-start regression test built specifically to check that a fresh entry must not score zero, and it was fixed by changing the term to $\log(2 + \text{freq})$, which preserves the same concave, increasing shape while guaranteeing a strictly positive score at zero frequency. The fix in Equation 1 was applied throughout every experiment in this report; it is disclosed here because a report that hid this detail would be less convincing about the correctness of what is actually being measured, not more.

Because the gate already refreshes anything past its cluster’s time-to-live before it can be served, the decay term inside the eviction value function is not itself performing staleness prevention; it acts as a soft prior that favors entries with more remaining useful life among the candidates the gate has already judged fresh. This is the intended division of labor, not an oversight, and the ablation in Section 5.2 tests it directly.

Eviction remains entry-count budgeted throughout, matching GPTCache’s own convention. Entry size in bytes is recorded and used only inside the value function’s denominator; no separate byte-budget eviction loop was built, which keeps a second subsystem off the critical path and is not required by anything in this project’s claims.

3.2 Cut scope

An earlier version of this design included a third component, calibrated per-cluster similarity thresholds, replacing GPTCache’s single global cosine threshold with one learned per cluster. It was cut before implementation began. Per-cluster thresholding turns out to be a lossy special case of two things that already exist in the literature, vCache’s per-embedding threshold learning and Category-Aware caching’s per-category treatment, so the cost of building and calibrating a third, weaker version of an idea already covered twice exceeded what it would have contributed to this project’s specific claim. It is listed as future work rather than pursued here.

4 Experimental setup

4.1 Workload

All experiments in this report run on W1, a deterministic, seeded synthetic workload generator built for this project. A second workload, derived from Wikipedia revision history with staleness ground truth taken from edit timestamps, was investigated in a timeboxed feasibility spike before any other implementation work began. That spike sampled thirty question-answer pairs from the SQuAD 2.0 development set[11], pulled each answer’s revision history from the public Wikipedia API, and applied an automatic invalidation rule comparing each revision’s answer sentence to the one before it. Twenty cases were hand-labeled by a human reader and compared against the automatic rule’s output; agreement was 40 percent, well short of the 80 percent

threshold this project’s plan required before committing to building a full loader. All twelve disagreements traced to the same failure mode: the rule flagged a sentence as changed when a copyedit, a template artifact, or an unrelated inserted sentence altered the surrounding text without touching the fact the question actually asked about. Reaching a reliable rule would require a proper wikitext renderer and paragraph-level alignment across revisions rather than a plain-text sentence comparison, estimated at eighteen to twenty-three additional hours, which the project’s schedule did not allot. The decision to proceed with W1 alone, made once and not revisited during the remainder of the project, is disclosed here as a limitation on external validity: every staleness result in this report comes from a workload this project itself generated, not from an independently sourced ground truth.

W1 generates queries across several streams inside each of a configurable number of topic clusters: canonical queries that establish a ground-truth answer, near-duplicate paraphrases of a canonical query sharing its answer identity, novel long-tail queries that never repeat, and time-shifted repeats of an existing answer. Every cluster carries its own randomly drawn half-life, from which a true decay rate follows, and its own regeneration-cost distribution. The generator guarantees that for every answer identity that repeats, at least one later occurrence lands after that answer’s true expiry time, since without that guarantee the validity gate would have nothing to reject in the trace at all. Each generated trace is split by arrival time into the first thirty percent, used only for calibration, and the remaining seventy percent, used only for evaluation; no parameter is ever fit on data it is later scored against.

The generator’s two ground-truth distributions, regeneration cost and content half-life, were meant to be calibrated against the Azure LLM Inference Trace and against observed update frequency in a public news or product-catalog corpus. Neither dataset was reachable from the development environment used to build this project, and rather than claim a fit that was never performed, both distributions are instead parameterized from commonly cited public figures for the quantities involved and compared against those figures as a plausibility check, documented plainly as such rather than presented as a real external fit. This is a second, explicitly disclosed limit on the workload’s external validity.

4.2 Simulation model

The benchmark harness that every experiment below runs through is a pure trace replayer. No language model is ever called. On a miss, the response content is whatever answer identity the trace recorded at generation time, the cost charged is the trace’s recorded regeneration cost, and the latency of that miss is simulated: drawn from a seeded log-normal distribution scaled by the entry’s recorded size in bytes, deterministic per query so that replay order never affects a query’s simulated cost. This distribution is a documented placeholder, not fit to any real inference trace, since fitting one was out of scope for this project’s schedule. Hit latency and the extension’s own added overhead, cluster lookup plus the gate check, are measured for real with wall-clock timing on the machine running the benchmark, not simulated. Reported latency in every results table is therefore a hybrid of real cache-side measurement and simulated backend cost; this is the right trade for making every run free, deterministic, and safe to run in continuous integration, but it is a validity limitation that must be stated plainly rather than buried, which is why it is repeated here rather than only in a code comment.

A further limitation shapes every absolute number in every table below. The benchmark harness’s index is an exact-text-match lookup; no embedding-based semantic index has yet been wired into the harness. This means paraphrase queries, one of W1’s four intentional stream types, never register as hits in these experiments, only literal repeats of an identical query string do. Hit counts across every experiment in this report are in the low hundreds out of many thousands of scored queries, and every confidence interval below is correspondingly wide. This caps the absolute hit rate reported everywhere in this document well below what a real semantic index would produce, but it applies identically across every condition in every

experiment, so it does not favor one policy, eviction scheme, or lambda source over another; the relative comparisons this report’s claims rest on are unaffected by it, even though the absolute numbers are not representative of a production semantic cache.

The cache starts empty in every run. The first ten percent of the evaluation split is replayed to warm the cache and excluded from every reported metric; only the remaining ninety percent of the evaluation split is scored.

4.3 Metrics and statistics

Stale-hit-rate is the fraction of served hits where the serve time exceeds the entry’s true, ground-truth expiry time; false-hit-rate is the fraction of served hits where the served answer’s identity does not match the querying question’s true answer identity. Cost saved is the sum of regeneration cost over non-stale hits only, since a stale hit served a wrong answer and saved nothing, and counting it as savings would overstate the extension’s benefit. Every configuration reported below was run with ten independent seeds, each producing its own freshly generated trace rather than replaying one fixed trace ten times, since the pipeline itself has no randomness once a trace is fixed and a repeated trace would produce ten identical rows with nothing to bootstrap a confidence interval over. Results are reported as medians with 95 percent confidence intervals from a percentile bootstrap over the ten per-seed values, ten thousand resamples, using a fixed seed for the resampling itself so the reported intervals are exactly reproducible. Group comparisons use the Mann-Whitney U test; the word “significant” is used in this report only where that test was actually run and its result is reported alongside it.

5 Results

5.1 Bracketing: does learned staleness beat naive pooling

The first and most consequential experiment tests the core claim behind fitting a staleness rate per cluster at all: does it actually do work relative to the two obvious alternatives, one shared rate pooled across every cluster, and the true rate a real system could never observe. Three lambda sources, global, learned, and oracle, were each run for ten seeds on the same twelve-thousand-query W1 configuration, with the gate enabled and FreCoS eviction active throughout.

Table 1: Bracketing experiment: median stale-hit-rate by lambda source, 10 seeds each, with 95% bootstrap confidence intervals.

Lambda source	Median stale-hit-rate	95% CI	Median cost saved (USD)
Global	0.144	(0.085, 0.214)	0.448
Learned	0.035	(0.024, 0.060)	0.449
Oracle	0.035	(0.028, 0.064)	0.449

Learned separates cleanly from global. Table 1 shows a median stale-hit-rate of 0.035 under learned against 0.144 under global; the two 95 percent confidence intervals do not overlap, and a Mann-Whitney U test comparing them gives $p \approx 0.0005$. Pooling every cluster’s observations into one rate is measurably and significantly worse than fitting each cluster’s own rate.

Where this experiment departs from what the design anticipated is in the relationship between learned and oracle. The two are statistically indistinguishable: learned’s median stale-hit-rate equals oracle’s exactly, eight of the ten seed pairs are byte-identical between the two conditions, and a Mann-Whitney U test comparing them gives $p \approx 0.94$. Learned does not sit at a visible midpoint between the naive floor and the unreachable ceiling; it sits at the ceiling. The mechanism is not mysterious: at roughly five hundred fifty to six hundred fifty calibration

observations per cluster in this configuration, well above the fallback threshold of thirty observations, the per-cluster maximum-likelihood fit essentially recovers the true generating rate, leaving no daylight for learned and oracle to differ on.

A follow-up run tested whether that gap would open at a substantially smaller calibration sample, using a trace roughly seven times smaller, which drops calibration observations per cluster to the thirty-to-seventy range, an order of magnitude sparser than the original run. It does not open. Learned remains indistinguishable from oracle at this smaller scale as well ($p \approx 0.96$), while learned versus global remains significant ($p \approx 0.0015$). The reason traced back to a confound rather than a property of the staleness fit itself: shrinking the trace shrinks the calibration sample and the evaluation-split hit count together, since both come from the same underlying trace-size knob, and it is the evaluation-side hit count, not the calibration sample alone, that determines whether calibration noise ever surfaces in a measured stale-hit-rate. With single-digit-to-low-double-digit hit counts per run at this smaller scale, the ratio is too coarse to register the extra estimation noise even though the estimation itself genuinely is noisier at this sample size.

Figure 1 shows both runs on one axis. The finding this experiment establishes, stated plainly rather than reframed to match a result the design anticipated in advance: per-cluster fitting from observed data beats a single shared rate by a large and significant margin, and does so robustly, tracking a rate the system could never actually observe across roughly a tenfold range of calibration sample sizes. That is a stronger form of the underlying claim than the originally anticipated “learned lands strictly between the two brackets” would have been, even though the literal bracket shape anticipated in advance did not materialize.

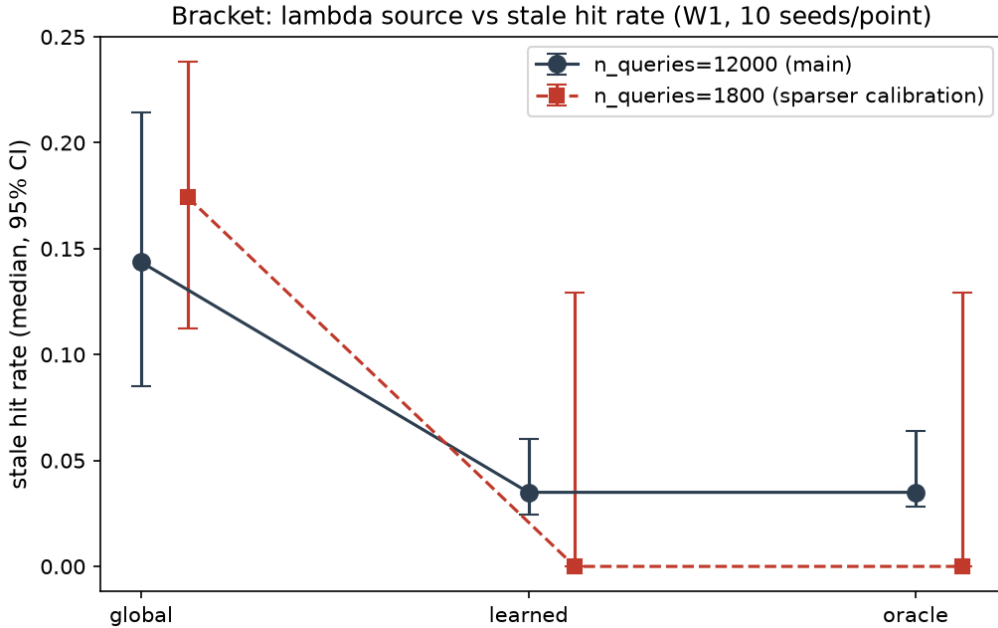


Figure 1: Stale-hit-rate under global, learned, and oracle lambda sources, at the original calibration scale and at a roughly sevenfold sparser calibration scale. Learned tracks oracle closely at both scales and separates significantly from global at both.

5.2 Ablation: isolating the gate and the eviction term

Six configurations were run for ten seeds each on the same workload: stock LRU with the gate disabled, stock LFU with the gate disabled (the floor of comparison per Biton and Friedman’s own finding that LRU underperforms on semantic workloads), a documented substitute for Biton and Friedman’s own released policy with the gate disabled, the validity gate combined

with plain LFU eviction, the validity gate combined with full FreCoS eviction, and the validity gate combined with FreCoS eviction with the size-normalization term removed.

Table 2: Ablation: median stale-hit-rate and hit rate by row, 10 seeds each. Row 2 (LFU, gate off) is the floor of comparison.

Row	Configuration	Stale-hit-rate	Hit rate	Δ stale-hit-rate vs. row 2
1	LRU, gate off	0.661	0.048	−0.019
2	LFU, gate off (floor)	0.680	0.052	—
3	B&F substitute, gate off	0.680	0.052	+0.000
4	LFU, gate on	0.042	0.017	−0.638
5	FreCoS, gate on (full stack)	0.035	0.017	−0.645
6	FreCoS (no size), gate on	0.042	0.017	−0.638

Against the LFU floor, median stale-hit-rate 0.680, the gate collapses stale-hit-rate by roughly fifteen times in every gated row (Table 2): gate-plus-LFU reaches 0.042, the full stack reaches 0.035, and the size-ablated variant reaches 0.042. Hit rate drops by roughly a third in the same rows, from 0.052 to about 0.017, which is the trade-off this project’s design anticipated in advance: the gate converts what would otherwise be stale hits into misses by construction, and that necessarily lowers the hit rate.

One row produced an unplanned but informative finding rather than the intended comparison. The documented substitute for Biton and Friedman’s policy turned out to be identical to plain LFU in every correctness and cost column, seed by seed, to full floating-point precision. This traced to a real property of the workload rather than a bug: at this configuration’s roughly five percent hit rate, almost every entry present at eviction time has never been accessed a second time, so the substitute’s frequency-divided-by-cost score evaluates to zero for essentially every candidate regardless of its cost, and the policy falls through to the same tie-break rule LFU uses when every frequency is tied at zero. The primary eviction comparator is therefore effectively untested by this particular workload’s parameters, a limitation stated here rather than concealed behind a coincidence-shaped result.

The interaction this project’s design predicted in advance, that the gate would carry most of the stale-hit-rate improvement with the eviction decay term contributing less on top of it than it contributes alone, held, and held more strongly than anticipated. Isolating the gate’s own effect (gate-plus-LFU against the LFU floor) gives a change of −0.638; isolating FreCoS’s decay term on top of an already-active gate (full stack against gate-plus-LFU) gives a further change of only −0.007, an order of magnitude smaller and not statistically distinguishable from zero ($p \approx 0.97$ by Mann-Whitney U, against $p \approx 0.0002$ for the gate’s own effect). The gate accounts for essentially the entire improvement, roughly ninety-nine percent of the total move from floor to full stack. This is not the contradiction the plan for this project flagged as worth investigating in advance, since the predicted direction held; it is a flatter result than even the design anticipated, and both the gate and the eviction value function were checked for a shared cluster-identity bug as a precaution regardless, since that specific integration failure mode was called out in advance as the most likely one, and neither component was found to disagree with the other about which cluster an entry belongs to.

The size-normalization row was likewise statistically indistinguishable from the full stack and from gate-plus-LFU on every metric. With the gate already filtering out anything stale, the entries the eviction policy actually chooses among are uniformly fresh, and the decay term inside the value function has little left to differentiate on; the size term’s effect is plausibly more visible under tighter cache-size pressure than this ablation’s fixed budget provides, which is exactly what the cache-size sweep below investigates.

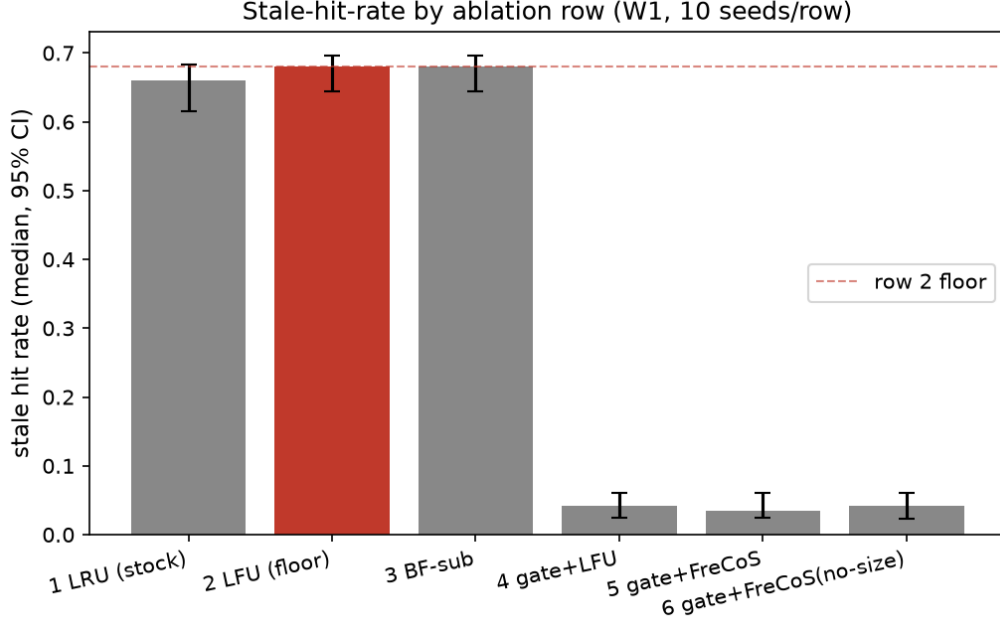


Figure 2: Stale-hit-rate across all six ablation rows, with 95 percent confidence intervals. Row two, LFU with the gate disabled, is marked as the floor of comparison.

5.3 Cache-size sweep and the correctness-efficiency trade-off

Cache size was swept across five points spanning five to eighty percent of the workload’s distinct-answer working set, full stack throughout. Hit rate rises from 0.013 at the smallest point to 0.017 at thirty percent of the working set, then flattens completely; the three largest points produce identical per-seed hit counts, query by query. The knee in this curve, identified by the point where the marginal hit-rate gain per additional cache entry drops by more than an order of magnitude relative to the previous step, falls at thirty percent of the working set: the slope between the second and third points is roughly thirteen times smaller than between the first and second, and the slope from the third point onward is exactly zero. This cache size is used as the operating point for every other sweep in this report.

The trade-off curve this project’s design describes as its honest core is the one produced by sweeping the gate’s target confidence level while holding cache size fixed at the knee identified above.

Table 3: TTL confidence sweep: median stale-hit-rate and hit rate as target confidence varies, 10 seeds each.

TTL confidence	Median stale-hit-rate	Median hit rate
0.80	0.126	0.019
0.90	0.042	0.017
0.95	0.023	0.017
0.99	0.008	0.016

As confidence rises from 0.80 to 0.99 (Table 3), stale-hit-rate falls from 0.126 to 0.008, a sixteenfold reduction, while hit rate falls from 0.019 to 0.016, a fifteen percent reduction. The exchange rate across this range is not uniform: most of the stale-hit-rate benefit is captured in the first step alone, from 0.80 to 0.90, where stale-hit-rate falls from 0.126 to 0.042, while hit rate erodes at a comparatively steady pace across the full range. The cheapest correctness gains come early; pushing confidence toward 0.99 buys a shrinking further reduction in staleness for

roughly the same ongoing cost in hit rate. This is presented as a trade-off curve, not as evidence that a stricter gate is uniformly better: an operator choosing a confidence level is choosing a point on this curve to suit their own tolerance for stale answers against their own tolerance for cache misses, not selecting a dominant option.

A fourth sweep varied the number of clusters at five, ten, twenty, and fifty. This axis came out non-monotone on both hit rate and stale-hit-rate, and every adjacent pair of confidence intervals overlaps heavily, so no directional effect is distinguishable from noise at ten seeds per point. A plausible mechanism is two effects pulling in opposite directions at comparable strength: more clusters means each one’s calibration sample shrinks, which should make the learned rate noisier, while at the same time finer-grained clusters could in principle track heterogeneous half-lives more precisely. This result is reported in a supplementary table rather than promoted to a headline figure, since it clears neither the effect-size nor the confidence-interval bar the rest of this report’s figures were held to.

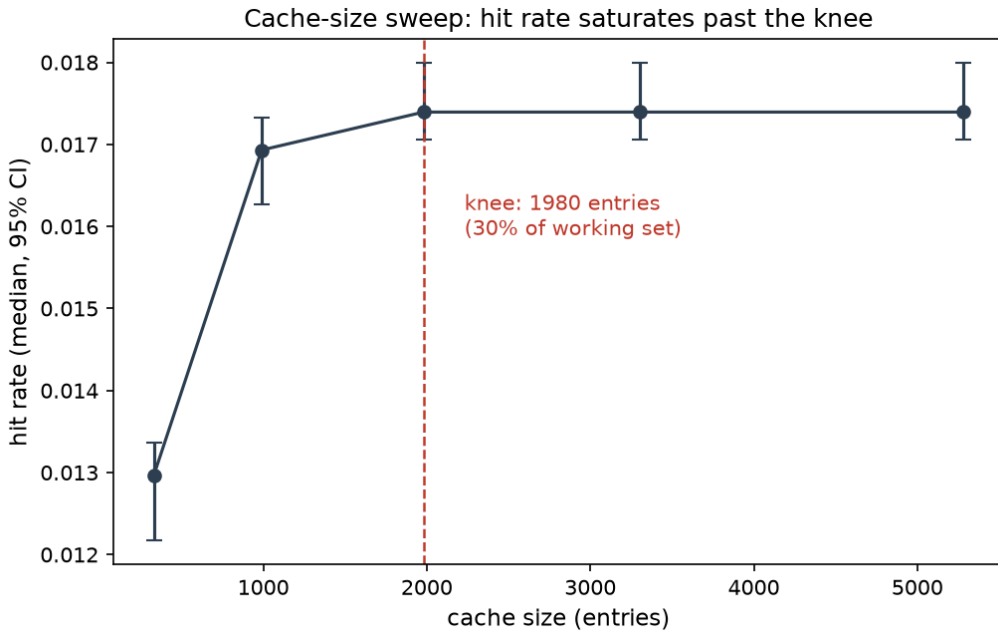


Figure 3: Hit rate against cache size, with the identified knee at thirty percent of the working set marked.

6 Discussion

The headline claim this project set out to test, that fitting a staleness rate from observed data does measurable work beyond either ignoring cluster structure entirely or assuming a single global rate, is supported clearly and repeatedly: the learned rate beats naive pooling with a large, statistically significant margin in every experiment that varied the lambda source, and does so whether calibration data is abundant or an order of magnitude sparser. The form that support takes, learned tracking oracle rather than sitting at a visible midpoint, is a stronger result for the underlying claim than the originally anticipated shape, even though it does not match the literal bracket picture the design predicted in advance; reporting it as the actual finding rather than reframing it to match the prediction is a deliberate choice, since the alternative would misrepresent what the data show.

The second finding, that the validity gate rather than the eviction value function’s decay term carries almost the entire measured stale-hit-rate improvement, is consistent with how this project’s design describes the two components’ intended division of labor: the gate is the

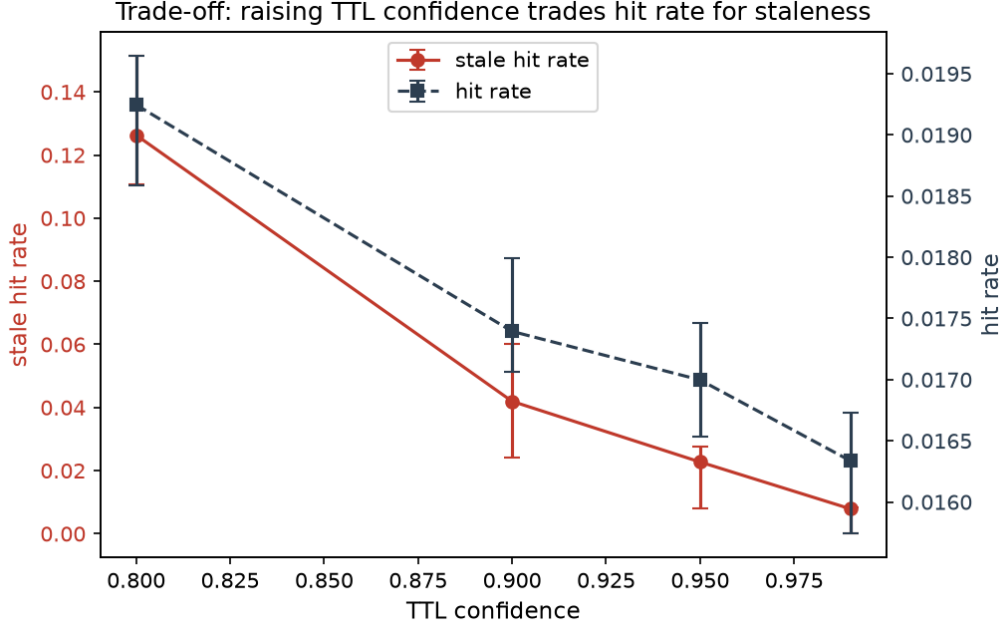


Figure 4: Stale-hit-rate and hit rate plotted together against the gate’s target confidence level, showing the shape of the correctness-efficiency exchange rate rather than a single winning configuration.

correctness mechanism, and the eviction term is meant only as a soft prior among entries the gate has already judged fresh. The ablation confirms that division holds in practice, more sharply than the design itself anticipated. This has a direct implication for anyone considering which piece of this system to adopt in isolation: the gate is doing essentially all of the correctness work measured here, and a system that wanted only the staleness-correctness gain without the added eviction complexity could plausibly adopt the gate alone.

The correctness-efficiency trade-off in Section 5.1 is the part of this report that most needs to be read as a trade-off and not a win. Raising the gate’s target confidence level reliably buys a large reduction in stale-hit-rate at a comparatively small, but real, cost in hit rate, and that exchange rate is front-loaded: most of the correctness gain is available cheaply, and further gains cost proportionally more in foregone hits. A system that adopts this gate is choosing a point on this curve according to how much a stale answer costs relative to how much a cache miss costs, and this report does not claim there is a universally correct point to choose.

Two limitations bound how far these results generalize, and both are stated here rather than left implicit. First, every result in this report comes from a synthetic, self-authored workload; the attempt to source an external, independently verified staleness ground truth from Wikipedia revision history was made in good faith and abandoned on measured evidence, forty percent rule agreement against an eighty percent bar, not abandoned for convenience, but the limitation stands regardless of the reason. Second, the benchmark harness’s exact-match index caps every absolute hit rate and stale-hit-rate number in this report well below what a semantic index would produce, and every finding above is a relative comparison across conditions that share this limitation identically, not a claim about absolute production-scale numbers. A reader should treat every rate in this report as directionally informative and every absolute value as an artifact of this project’s specific harness rather than a production estimate.

7 Conclusion and future work

This project set out to name a gap in how semantic caches are evaluated, stale-hit-rate, and to build and measure a system that closes it without abandoning cost-awareness in eviction.

The gate reliably converts most stale hits into misses at a bounded and quantified cost in hit rate; the learned per-cluster staleness rate that feeds it outperforms naive pooling significantly and consistently, tracking an unreachable oracle rate closely across an order of magnitude of calibration sample sizes; and the eviction value function’s cost and size terms behave correctly under every property test built for them, even though their measured marginal contribution on top of an already-active gate is smaller than this project’s own design predicted in advance.

Several directions were deliberately set aside rather than pursued, and are recorded here as grounded future work rather than speculative padding. A real semantic index in place of the exact-match harness used throughout this report would very likely change every absolute number reported above and is the single change most likely to make this system’s results comparable to a production deployment; the relative findings, learned beating pooled, the gate carrying most of the correctness gain, are expected to be directionally robust to that change, but this project did not verify that expectation. Calibrating the workload generator’s cost and half-life distributions against a real inference trace and a real content-update corpus, rather than the literature-informed proxies used here, would strengthen the external validity this report currently discloses as a limitation. Dynamic re-clustering under drift, a probabilistic staleness budget in the style of FreshCache as an alternative to this project’s hard gate, and admission control layered on top of the current eviction policy were all named as future work in this project’s original design and remain so.

Appendix: code and data artifacts

All code, tests, and results referenced in this report are in the project’s git repository. The extension package lives under `gptcache_ext/`, split into the serve-path pipeline seam (`pipeline.py`), additive metadata storage (`metadata.py`), the staleness model (`staleness/clusters.py`, `staleness/fitter.py`, `staleness/gate.py`), and eviction (`eviction/frecos.py`, `eviction/baselines.py`). The synthetic workload generator is under `workloads/w1_synthetic/`. The benchmark harness and its metrics are under `benchmarks/harness.py` and `benchmarks/metrics.py`; the four experiment runners referenced in Section 5 are under `benchmarks/runners/`. Every experiment’s raw results and its own written summary are under `results/`, organized by experiment (`results/brackets/`, `results/ablation/`, `results/sweeps/`). Every figure in this report is regenerated from those results by a single command,

```
cd analysis && python3 make_figures.py
```

with no hand-edited figure anywhere in this project. The reference oracle used to validate the pipeline’s decision logic and the callable invariant suite run on every benchmark execution are under `tests/oracle/` and `tests/invariants.py`. The GPTCache commit this project vendors and every source-code claim made about its behavior in Section 2 are recorded in `docs/baseline-source-map.md`. The Wikipedia feasibility spike discussed in Section 4.1 is recorded in full in `docs/w2-feasibility.md`.

References

- [1] N. Biton and R. Friedman. *From Exact Hits to Close Enough: Eviction Policies for Semantic Caches*. arXiv:2603.03301, 2026.
- [2] Cortex authors. *Cortex: A Cost-Aware Semantic Caching System for LLM Serving*. arXiv:2509.17360, 2026. NSDI 2026; earlier preprint circulated as “Asteria”.

- [3] L. Cherkasova and G. Ciardo. *Role of Aging, Frequency, and Size in Web Cache Replacement Policies*. Proceedings of the International Conference on High-Performance Computing and Networking (HPCN), 2001.
- [4] Category-Aware authors. *Category-Aware Semantic Caching for Large Language Model Serving*. arXiv:2510.26835, 2025.
- [5] FreshCache authors. *FreshCache: Probabilistic Staleness Budgets for Retrieval-Augmented Generation*. arXiv:2607.04281, 2026.
- [6] SCALM authors. *SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models*. arXiv:2406.00025, 2024.
- [7] MeanCache authors. *MeanCache: User-Centric Semantic Caching for LLM Web Services*. arXiv:2403.02694, 2024.
- [8] G. Einziger and R. Friedman. *TinyLFU: A Highly Efficient Cache Admission Policy*. arXiv:1512.00727, 2015.
- [9] vCache authors. *vCache: Verified Semantic Prompt Caching*. arXiv:2502.03771, 2025. ICLR 2026.
- [10] Zilliz. *GPTCache: A Library for Creating Semantic Cache for LLM Queries*. <https://github.com/zilliztech/GPTCache>, 2024. Version 0.1.44, commit bae7ffef774e762d9d4e60fce70be00011188a6.
- [11] P. Rajpurkar, R. Jia, and P. Liang. *Know What You Don’t Know: Unanswerable Questions for SQuAD*. arXiv:1806.03822, 2018.